IMPACTA DIGITAL

Software Architecture & Clean Architecture

Prof. Eduardo Galego



Bio



Mestre em Ciências da Computação na área de IA pelo IME-USP, MBA em Soluções Corporativas Java pela FIAP e Bacharel em Ciências da Computação pelo UNASP.

Desde 2004 atua como programador e analista de sistemas. Também atua como professor universitário desde 2010, lecionando nos cursos de Graduação e MBA.

Atualmente é Engenheiro de Software Sênior no PagBank, professor de pós-graduação na FIAP e Impacta.

/in/efgalego

Conteúdo Programático

Esta disciplina combina os princípios fundamentais da arquitetura de software com as práticas específicas da Clean Architecture, proporcionando uma compreensão completa para o design de sistemas escaláveis, de alta performance e fácil manutenção.

O aluno será capacitado a aplicar diversos padrões arquiteturais, como microservices, monolitos e arquiteturas orientadas a eventos, além de aprender a separar responsabilidades em diferentes camadas do sistema utilizando a Clean Architecture. O foco será na criação de sistemas com baixo acoplamento, alta coesão e independência de frameworks, bancos de dados e outras dependências externas.

A disciplina também abordará os princípios de design como a inversão de dependências, a modularização do código, a utilização de testes unitários e a documentação de decisões arquiteturais. Ao final, o aluno será capaz de projetar e implementar arquiteturas que favoreçam a evolução contínua do software, mantendo a qualidade e a manutenção em longo prazo.



Conteúdo Programático



Divisão das Aulas

MÓDULO I

Introdução à Arquitetura de Software

1.1 Motivação
1.2 Paradigmas de Programação
1.3 Ciclo de Vida de Desenvolvimento de Software
1.4 Conceitos Básicos

MÓDULO II

Padrões Arquiteturais

2.1 Estilos de Arquitetura
2.2 DDD
2.3 Clean Architecture
2.4 Documentação de Decisões de Arquiteturais

MÓDULO III

Design

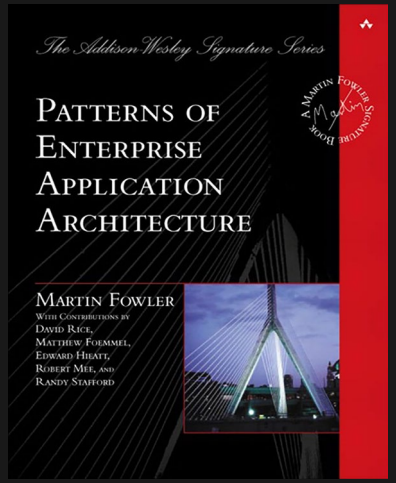
3.1 Conceitos
3.2 Princípios de Design
3.3 Padrões de Design
3.4 Testes
3.5 Conclusão



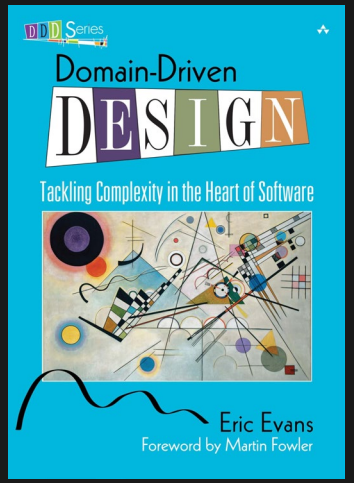
Bibliografia



MARTIN, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Addison-Wesley, 2017.



FOWLER, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.



EVANS, E. Domain-Driven Design: Tackling Complexity in the Heart of Software. Addison-Wesley, 2003.

Módulo I

Introdução à Arquitetura de Software

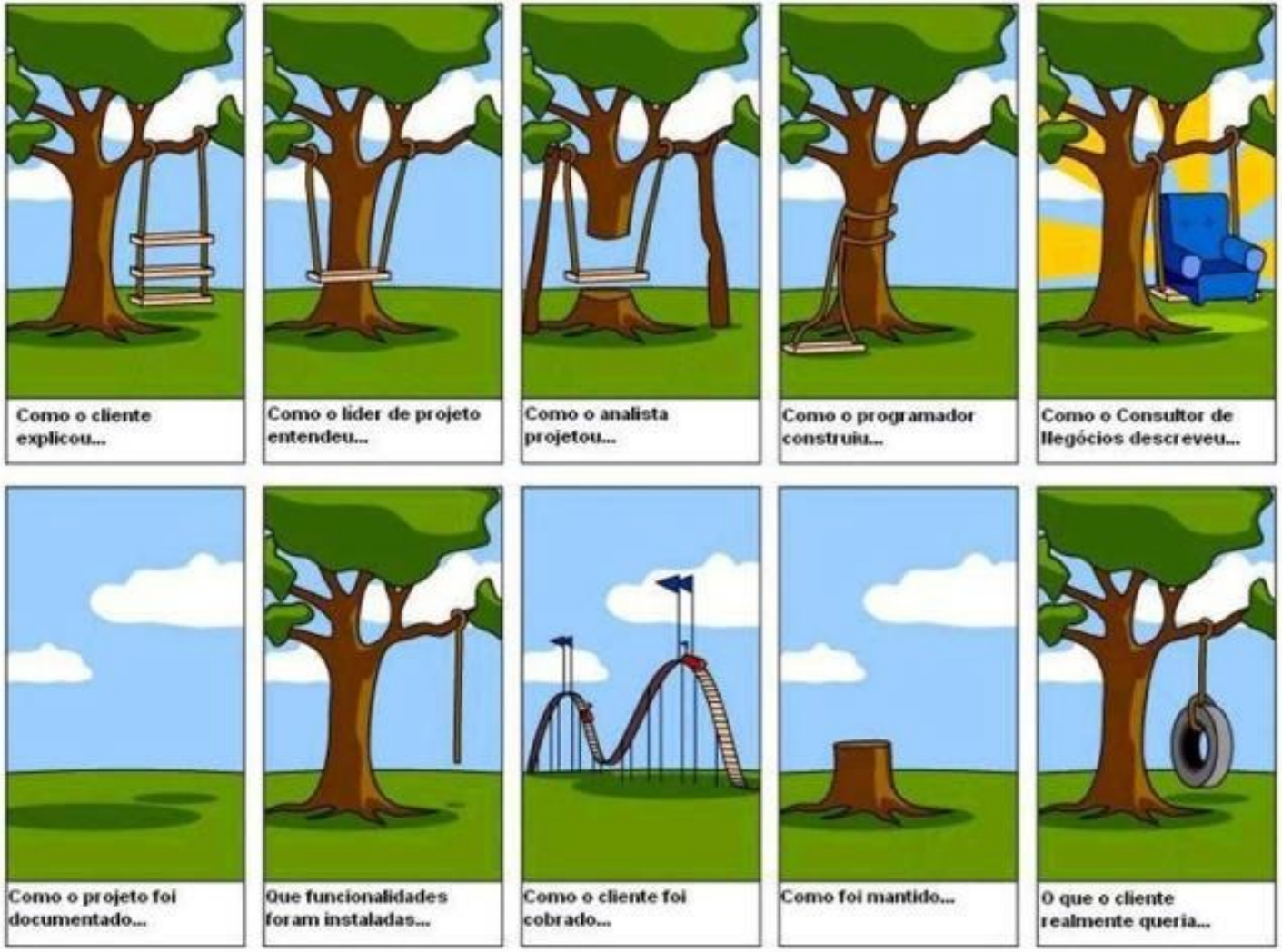


Motivação



Por que Projetos de Desenvolvimento de Software falham?

Extraído de https://www.freepik.com/free-vector/employee-angry-office_901998.htm



Extraído de https://flavioaf.wordpress.com/2011/09/20/conceitos-de-gerenciamento-de-projetos-parte-3/

Apenas 64% dos projetos atingem seus objetivos.

Extraído de https://www.projectmanagementworks.co.uk/project-failure-statistics/

“

39% de todos os projetos são bem-sucedidos, ou seja, entregues no prazo, dentro do orçamento e com os recursos e funções necessários.

”

Extraído de https://www.projectmanagementworks.co.uk/project-failure-statistics/

“70% do retrabalho era devido à falta de conhecimento do domínio durante as fases de requisitos e design.”

Extraído de https://simpleisbetterthancomplex.com/media/2016/10/chaos-report.pdf

Pequenos Projetos (menos que $1 milhão)	Grandes Projetos (maiores que $10 milhão)
76% foram bem sucedidos	10% foram bem sucedidos
20% foram desafiadores	52% foram desafiadores
4% falharam	38% falharam

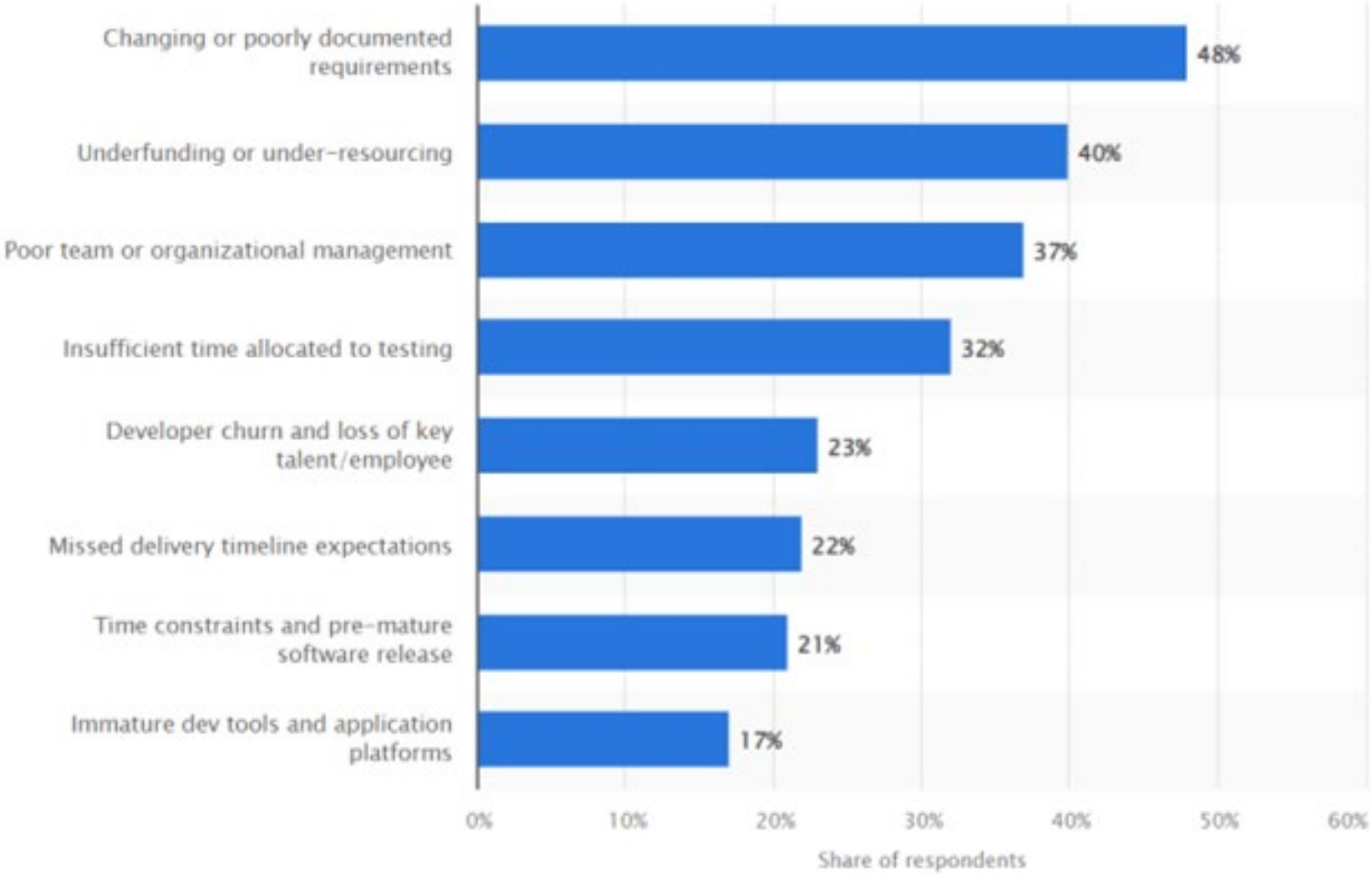
Extraído de https://www.projectmanagementworks.co.uk/project-failure-statistics/

“

Organizações perdem US$ 109 milhões para cada US$ 1 bilhão investido em projetos e programas.

”

Extraído de https://www.projectmanagementworks.co.uk/project-failure-statistics/



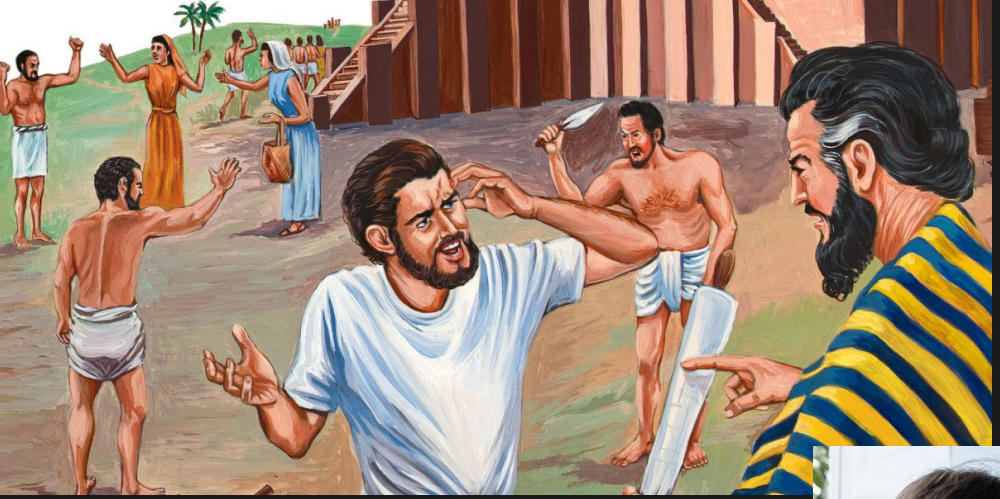
Extraído de https://www.requiment.com/why-do-software-development-projects-

Torre de Babel (Gênesis 11,1-9)

1 E era toda a terra de uma mesma língua e de uma mesma fala.
2 E aconteceu que, partindo eles do oriente, acharam um vale na terra de Sinar; e habitaram ali.
3 E disseram uns aos outros: Eia, façamos tijolos e queimemo-los bem. E foi-lhes o tijolo por pedra, e o betume por cal.
4 E disseram: Eia, edifiquemos nós uma cidade e uma torre cujo cume toque nos céus, e façamo-nos um nome, para que não sejamos espalhados sobre a face de toda a terra.
5 Então desceu o Senhor para ver a cidade e a torre que os filhos dos homens edificavam;
6 E o Senhor disse: Eis que o povo é um, e todos têm uma mesma língua; e isto é o que começam a fazer; e agora, não haverá restrição para tudo o que eles intentarem fazer.
7 Eia, desçamos e confundamos ali a sua língua, para que não entenda um a língua do outro.
8 Assim o Senhor os espalhou dali sobre a face de toda a terra; e cessaram de edificar a cidade.
9 Por isso se chamou o seu nome Babel, porquanto ali confundiu o Senhor a língua de toda a terra, e dali os espalhou o Senhor sobre a face de toda a terra.



Imagem extraída de https://mundoeducacao.uol.com.br/historiageral/torre-de-babel.htm



O que há em comum entre as duas cenas?



Imagem extraída de https://www.jw.org/pt/biblioteca/livros/historias-biblicas/2/torre-de-babel/

Imagem extraída de https://www.zoho.com/blog/pt-br/workplace/falha-de-comunicacao.html

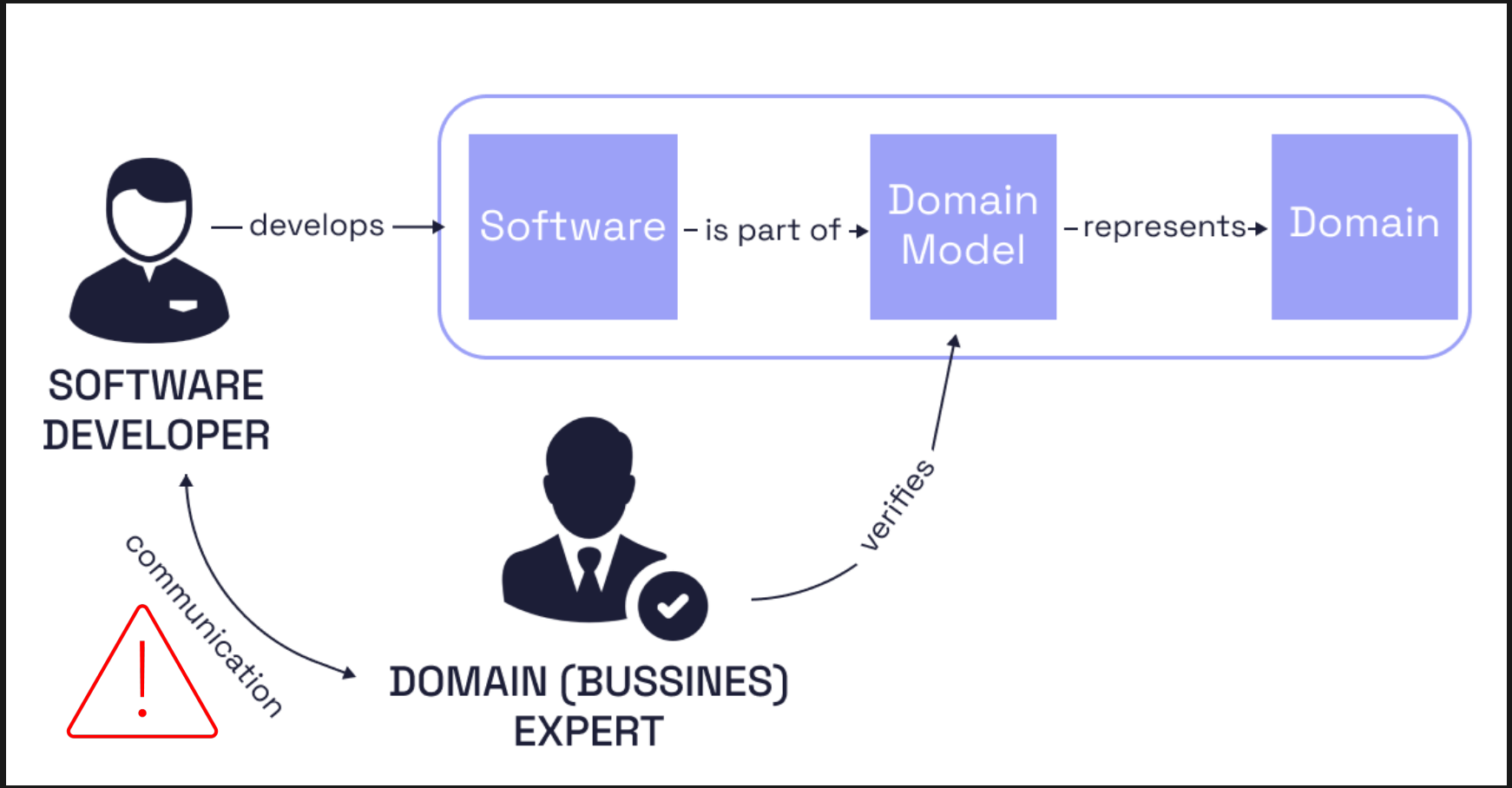


Imagem extraída de https://www.linkedin.com/pulse/domain-driven-design-why-do-companies-need-approach-1e/

```
function cal(a, b) {
  let r = a * b;
  return r;
}
```

OU

> “Escrevemos código não para máquinas, mas para outros programadores

```
function calcularTotalCompra(
                quantidadeItens, precoUnitario) {
  let totalCompra = quantidadeItens * precoUnitario;
  return r;
}
```

Exemplo adaptado de https://www.dio.me/articles/desvendando-o-clean-code-principios-exemplos-e-boas-praticas

~ Nosso Desafio ~

“Escrever código que expresse de maneira clara e concisa as regras de negócio, bem como regras implícitas de organização e boas práticas.”



~ Nosso Desafio ~



Extraído de https://br.freepik.com/fotos-gratis/interior-baguncado-cheio-de-roupas_152536950.htm



Extraído de https://www.decorfacil.com/quartos-de-meninos/


CI/CD
Acoplamento
Documentação
Qualidade
Orientação a Objetos
Design Patterns
Crescimento Horizontal
Imutabilidade
Fronteira
Crescimento Vertical
BDD
DevOps
Testes
TDD
APIs
Coesão
SOLID
Monitoramento
Microservices
Independência
Granularidade
Monolitos
Limite
Computação distribuída

IMPACTA DIGITAL

Conceitos

O que é Arquitetura de Software?

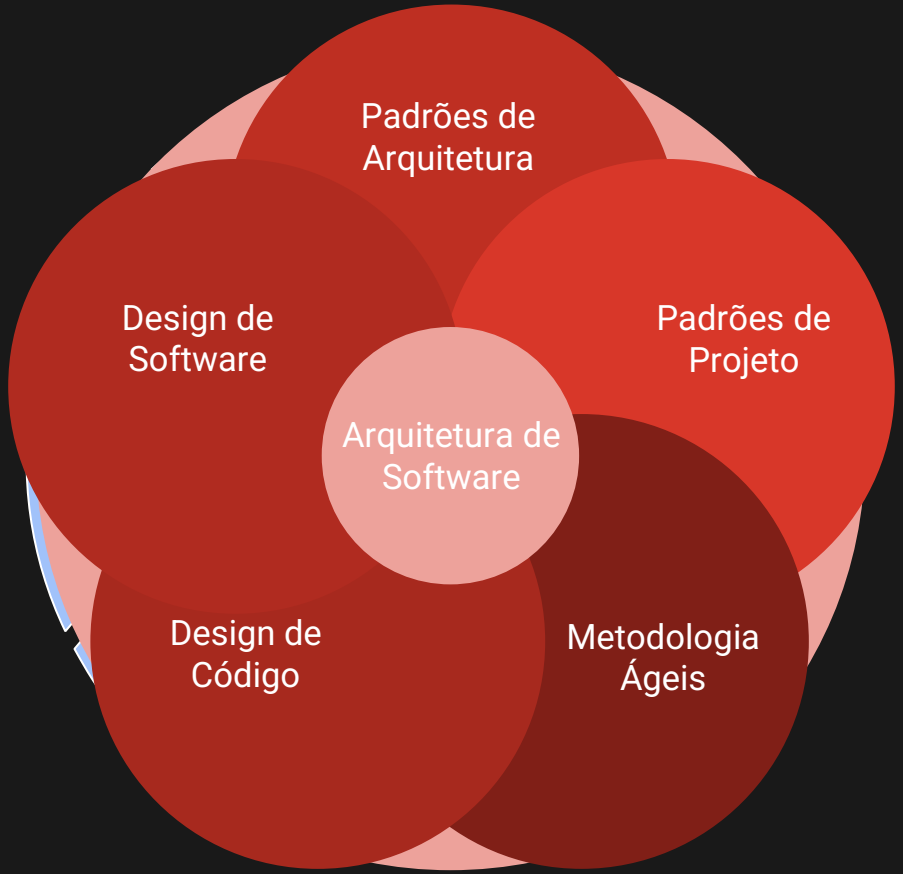


https://www.youtube.com/watch?v=DngAZyWMGR0



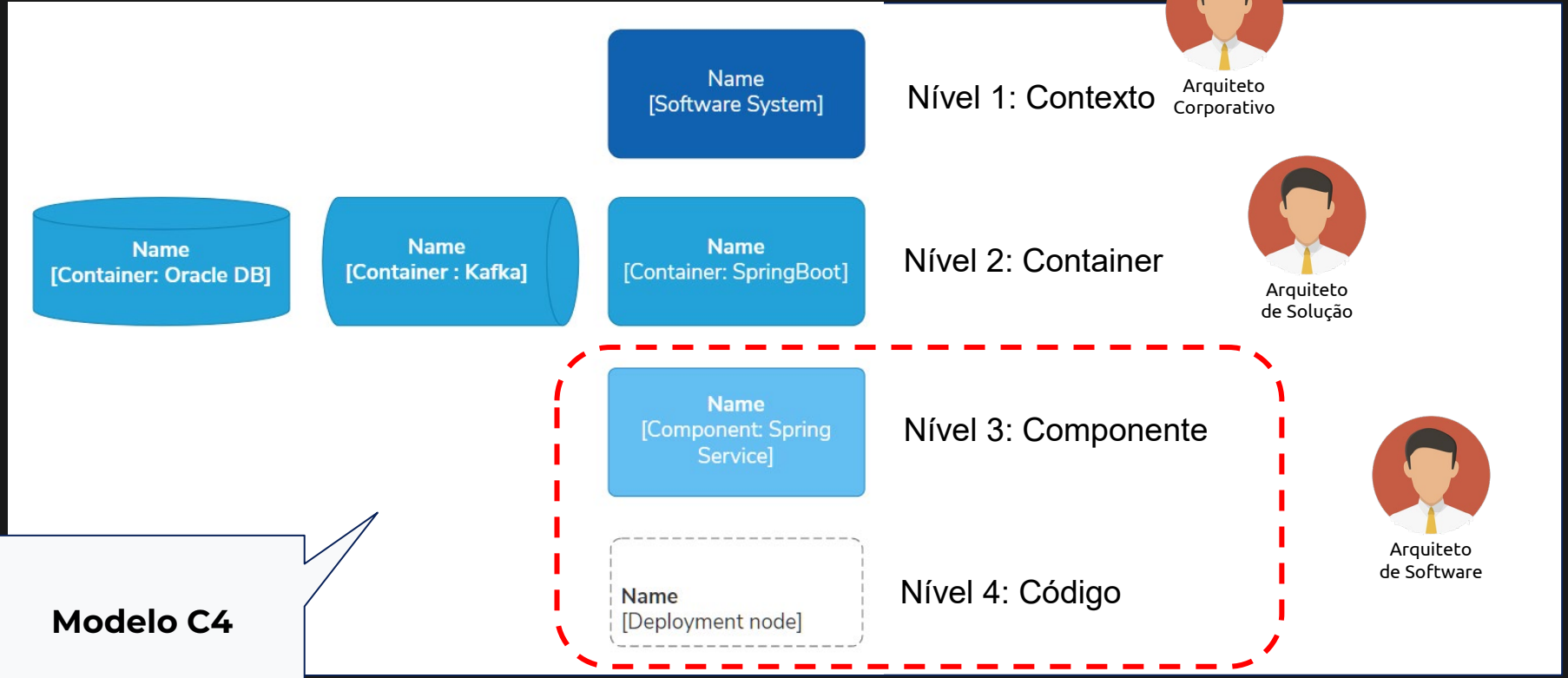
Conceitos

“Arquitetura de software são aquelas decisões que são importantes e difíceis de mudar”. Martin Fowler.



Adaptado de https://www.alura.com.br/artigos/padroes-arquiteturais-arquitetura-software-descomplicada

Conceitos



Extraído de https://www.softwarearchitecturemodels.com/c4model/

Conceitos

Computação



- Local onde o software é executado;
- Conhecido como CPU;
- Exemplos:
  - Servidor na nuvem;
  - Servidor dedicado;
  - Dispositivo mobile;
  - Navegador (web);
  - Dispositivo IOT;
  - Dispositivo Wareable.

Memória



- Onde o software armazena os valores das variáveis durante execução;
- Cuidado para não confundir com Storage (memória não volátil);
- Medido em bytes;
- Softwares que realizam mais cálculos podem requerer mais memória.

Rede

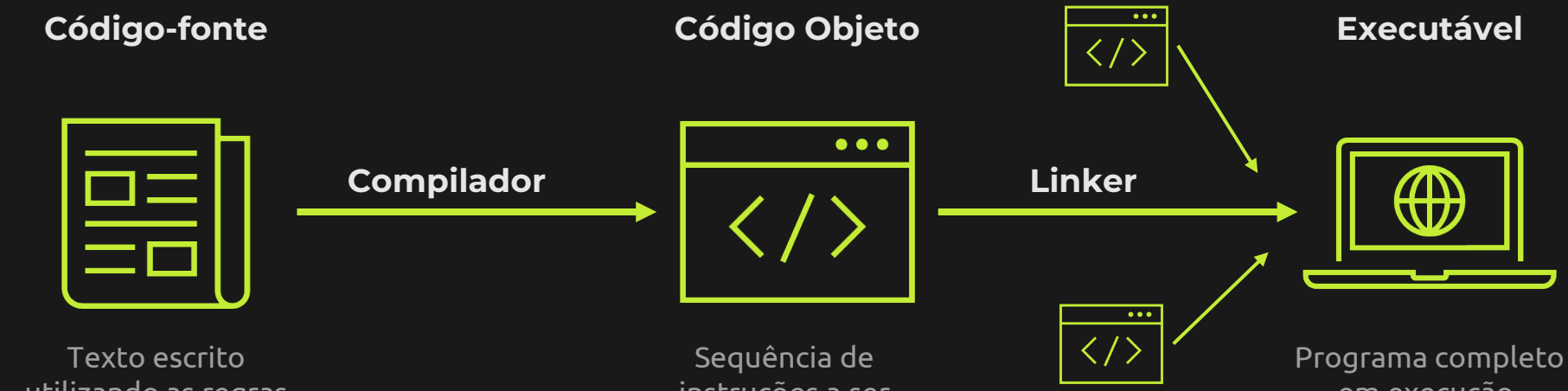


- Mecanismo de comunicação do software com usuário ou outros softwares;
- Medido em quantidade de bytes por segundo;

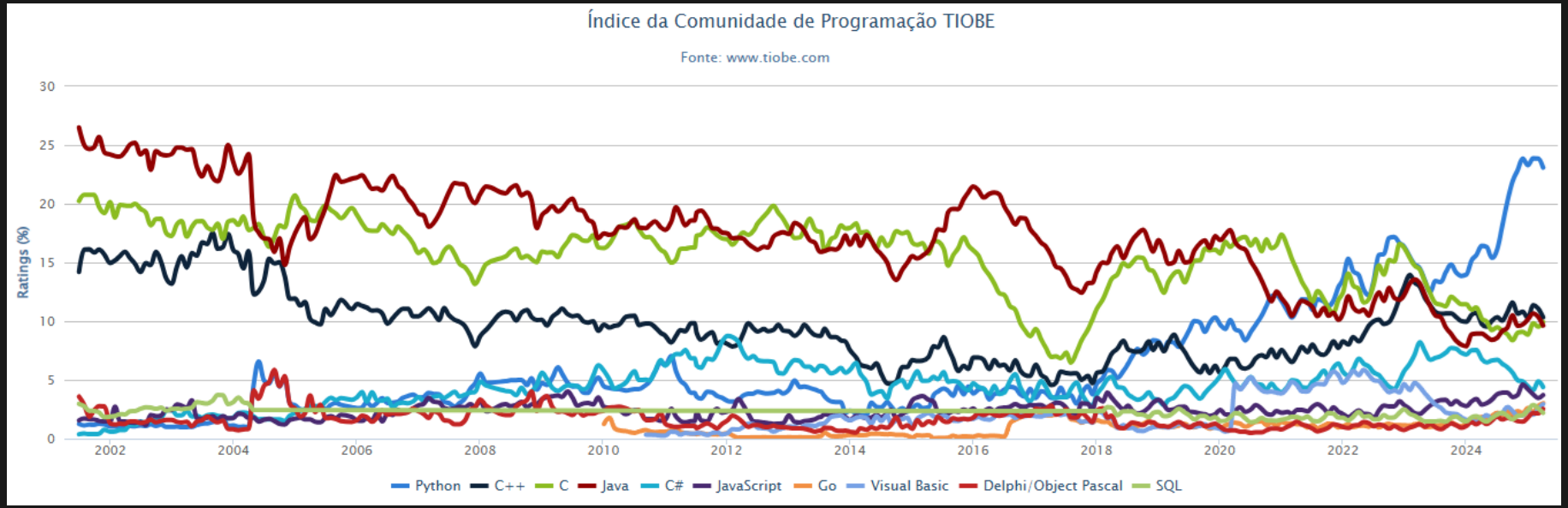
Monitoramento

Uso de CPU  Monitora o percentual de utilização comparado ao limite disponível	**Throughput**  Monitora a quantidade de requisições atendidas com sucesso	**Taxa de Erros**  Monitora a quantidade de erros que o software lança
Uso de Memória  Monitora o percentual de utilização comparado ao limite disponível	**APDEX**  Índice que monitora a satisfação do clientes, medido de zero a um	**Tempo de Atividade**  Medição do percentual em que a aplicação está disponível

Conceitos


Código-fonte
Compilador
Código Objeto
Linker
Executável
Texto escrito utilizando as regras de uma linguagem de programação de alto nível, ou seja, para melhor compreensão humana.
Sequência de instruções a ser executada pelo microprocessador.
Programa completo em execução.

Linguagem de Programação



www.tiobe.com

Paradigmas de Programação

Procedural

- Código executado em sequência;
- Organizado em funções e subrotinas;
- Principais conceitos:
  - Variáveis e escopo;
  - Estruturas (ex. struct);
  - Funções, argumentos, retorno e chamada;
  - Condicionais (ex. if);
  - Laços de repetição (ex. while);
  - ~~Go To~~;
  - Interfaces (header files).

Orientado à Objetos

- Código mais fragmentado, facilitando o reuso;
- Organizado em classes;
- Principais conceitos:
  - Classes, objetos e instância;
  - Funções acessores, modificadores e construtores;
  - Encapsulamento;
  - Herança;
  - Polimorfismo;
  - Abstração.

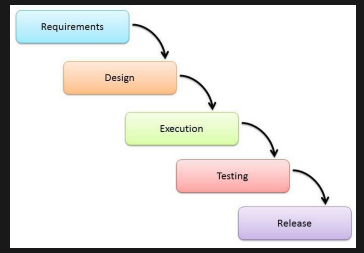
Funcional

- Código executado a partir do encadeamento de chamadas de funções;
- Utilizado em computação distribuída;
- Principais conceitos:
  - Funções puras;
  - Imutabilidade;
  - Composição de funções.

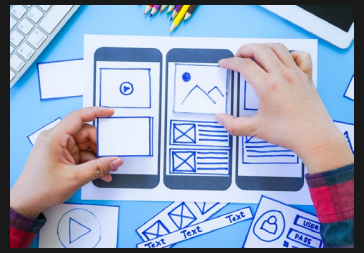


Ciclo de Vida de Desenvolvimento de Software

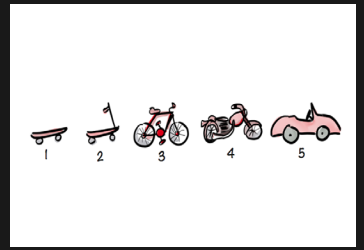




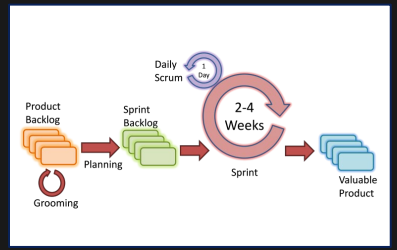
Waterfall



Prototyping



Incremental

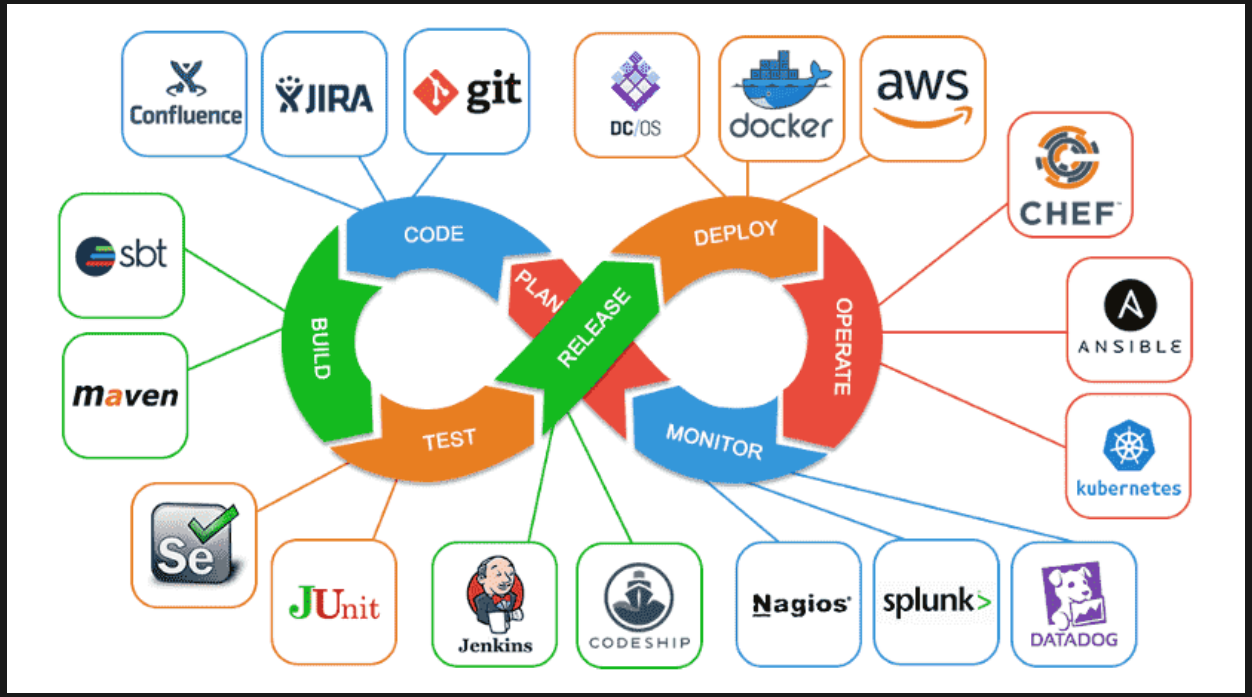


Agile

Extraído de https://melsatar.blog/2012/03/15/software-development-life-cycle-models-and-methodologies/

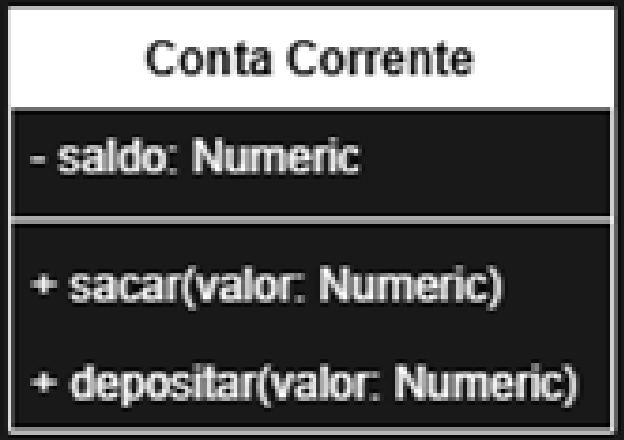
CI / CD

Descrevem a capacidade de entrega de versões do software dentro de fluxo contínuo, semelhante a uma esteira de linha de produção.



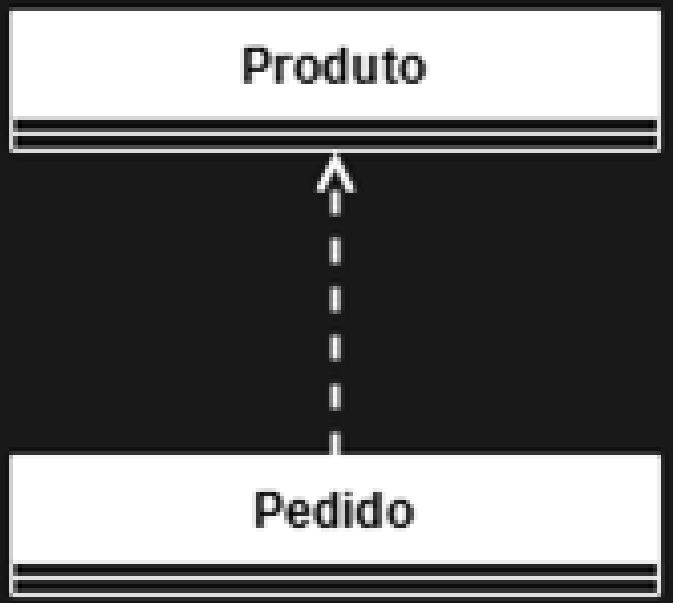
Extraído de https://blog.devgenius.io/what-is-ci-cd-concept-375cb226cf3d

Escopo



Significa a abrangência em que um componente ou variável está visível para acesso ou edição a partir de outro componente.

Dependência



Os componentes podem estar unidos por um relacionamento de Dependência.

Significa que mudanças no componente destino podem impactar no componente origem.

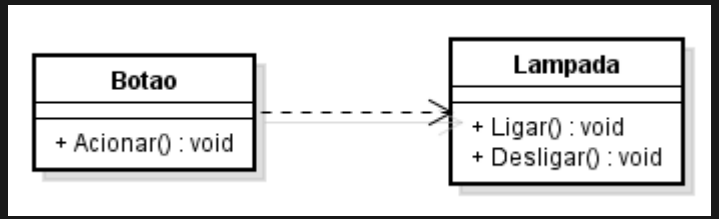
Inversão de Dependência



Vamos aprofundar este e outros princípios nas próximas aulas!

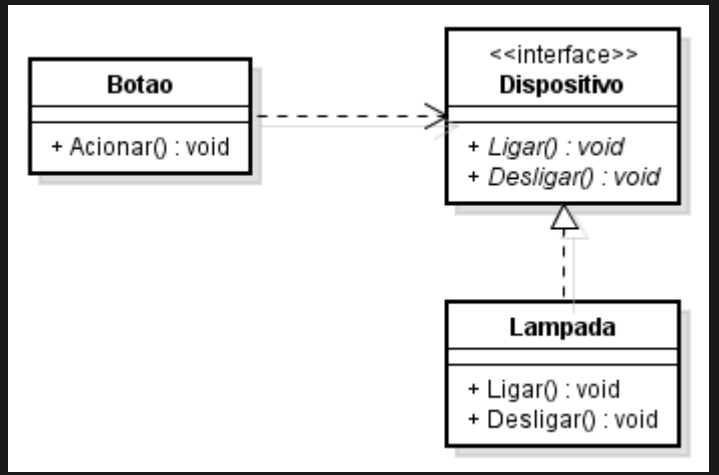
Princípio utilizado para inverter a dependência entre dois componentes.

Tradicional:



Botão não depende mais de lâmpada

Abstrato:

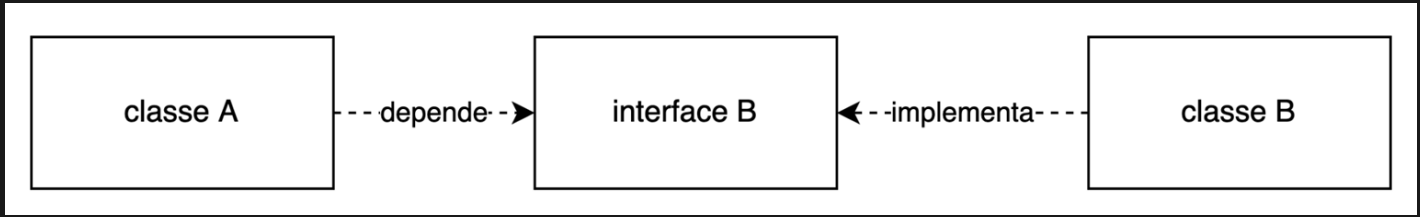


Extraído de https://robsoncastilho.com.br/2013/05/01/principios-solid-principio-da-inversao-de-dependencia-dip/

Acoplamento / Desacoplamento

Princípio que torna um componente independente / independente dos demais.

Trata-se de um conceito fundamental para trabalhar com Camadas e Microsserviços.



O desacoplamento induz complexidade, maior número de componentes e latência.

Extraído de https://dev.to/zanfranceschi/conceito-acoplamento-22c

Granularidade

Refere-se ao nível de detalhe ou abstração utilizado no design de software, especialmente no que diz respeito aos componentes e seus comportamentos

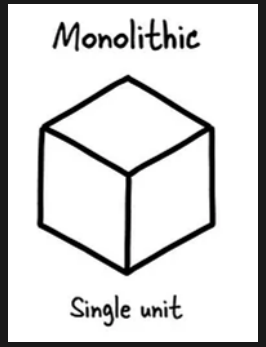
Granularidade Fina

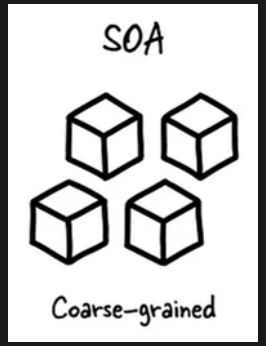
Componentes mais pequenos e detalhados podem ser mais fáceis de reutilizar, mas podem aumentar a complexidade da integração e do desenvolvimento.

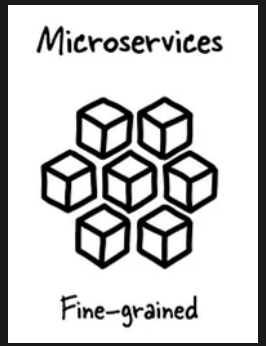
Baixa Granularidade

Componentes maiores e mais abstratos podem ser mais fáceis de manter e entender, mas podem limitar a flexibilidade do sistema e a reutilização de componentes.

Extraído de https://nadermedhatthoughts.medium.com/granularity-in-software-architecture-bec7c432d6d3



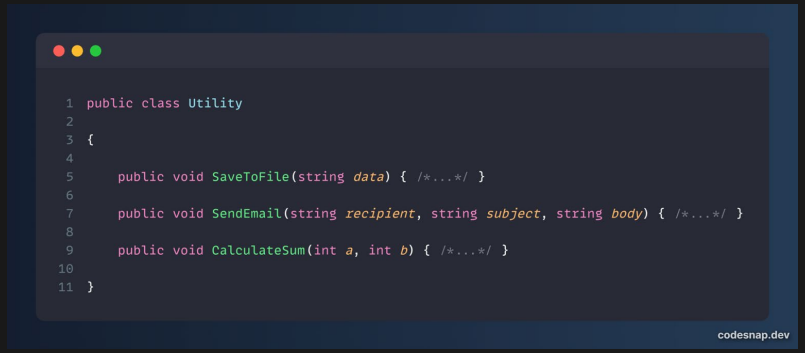




Coesão

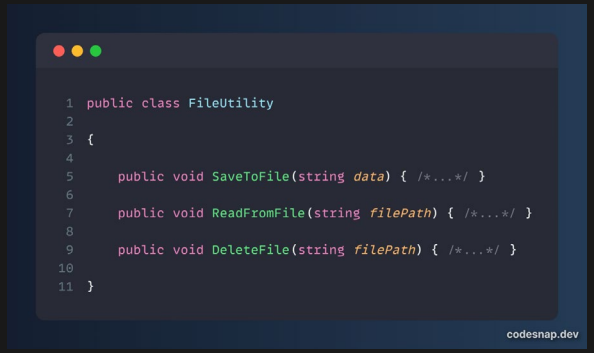
Refere-se ao grau em que os elementos de um módulo estão relacionados e trabalham juntos para realizar uma única tarefa.
Exemplo:

Baixa Coesão



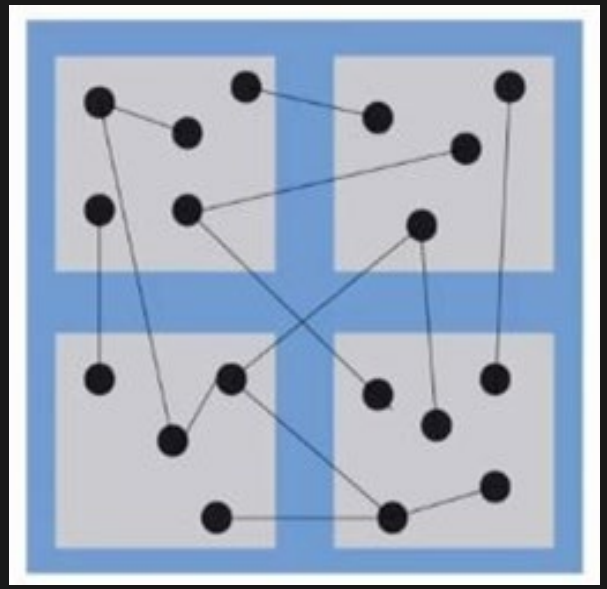
Os métodos dentro da classe Utility não têm nenhuma relação entre si e foram agrupados aleatoriamente.

Alta Coesão

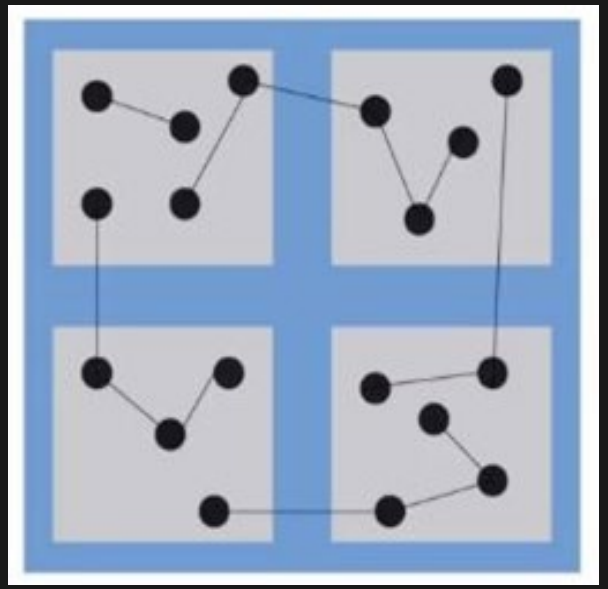


Todos os métodos da classe FileUtility estão relacionados ao manuseio de arquivos, formando um grupo lógico e coeso.

Extraído de https://www.linkedin.com/pulse/coes%C3%A3o-desenvolvimento-de-software-princ%C3%ADpios-e-rodrigo-de-oliveira-wj9tf/



OU



Alto Acoplamento
Baixa Coesão

Poucas classes e grandes
Mais dependências por classe
Maior conhecimento

Baixo Acoplamento
Alta Coesão

Muitas classes e pequenas
Menos dependências por classe
Menor conhecimento

Alvo a ser atingido!

Extraído de https://www.linkedin.com/pulse/acoplamento-e-coes%C3%A3o-em-software-anderson-de-oliveira-lima/

Escalabilidade

Capacidade de aumentar ou diminuir de modo rápido e fácil o tamanho ou a potência de uma solução de TI ou de um recurso

Vertical



Aumento ou redução é feito alterando os níveis de desempenho dos componentes atuais. Exemplo: Adicionar mais memória.

Horizontal



Aumento ou redução é feito incluindo novos nós, de modo que o cluster consiga reconhecer automaticamente e distribuir a carga igualmente.

IMPACTA DIGITAL

Módulo I

Estudo de Caso

Estudo de Caso

Como Iniciamos a Construção de uma Arquitetura de Software?



https://www.youtube.com/watch?v=343p-0E_9eE

Estudo de Caso

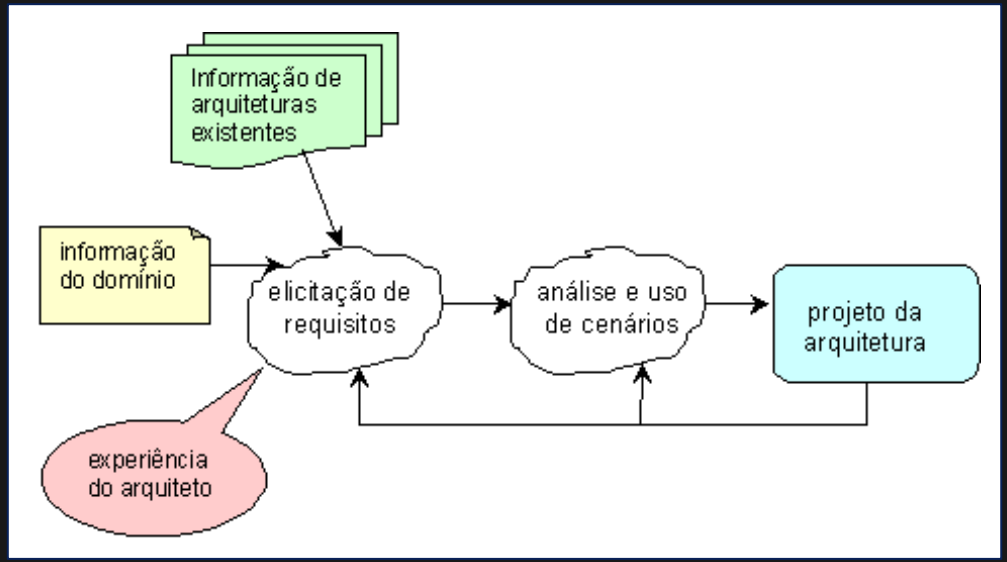
Requisitos *descrevem o que o sistema deve fazer e quais características são necessárias para atender às expectativas dos usuários.*

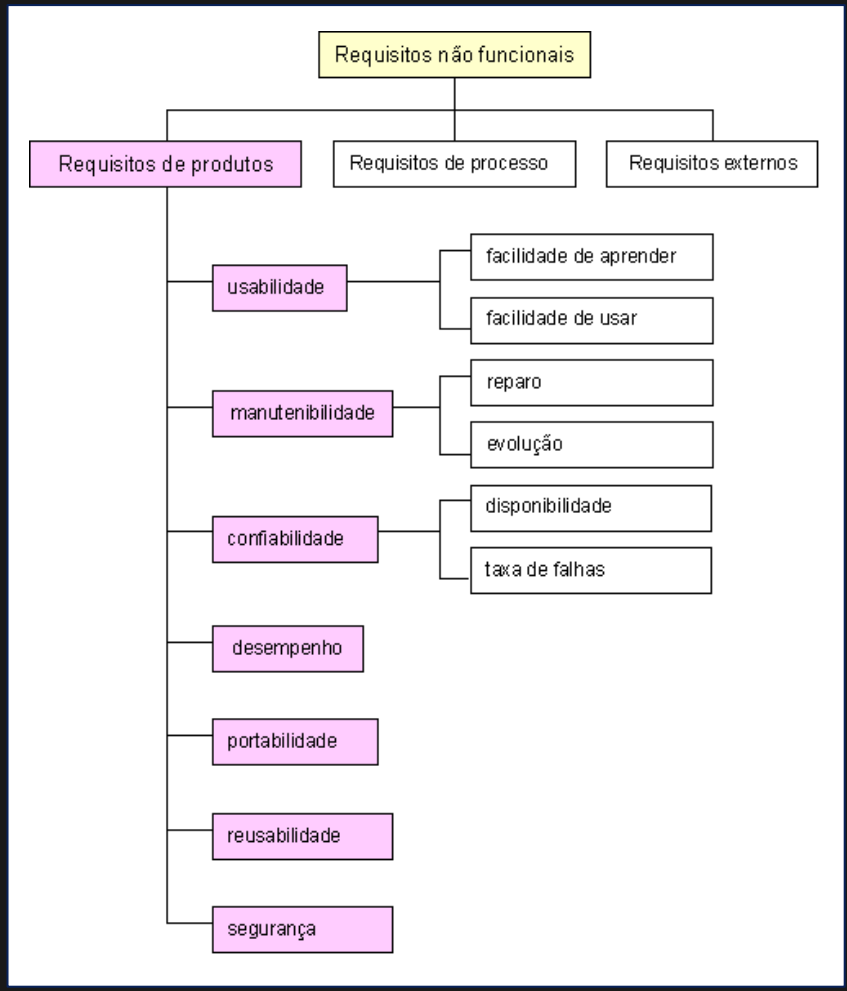
Requisitos de Arquitetura *são responsáveis por definir as funcionalidades e as características não funcionais do sistema.*

Eles estabelecem as necessidades do cliente, as restrições técnicas, as metas de qualidade e desempenho, bem como outras demandas relevantes para o projeto.

Adaptado de https://awari.com.br/requisitos-na-arquitetura-de-software-definindo-as-necessidades-funcionais-e-nao-funcionais-do-sistema/

Estudo de Caso





Extraído de https://www.devmedia.com.br/artigo-engenharia-de-software-3-requisitos-nao-funcionais/9525

IMPACTA DIGITAL

Revisão

- Por que estudar Arquitetura de Software?
- O que é Arquitetura de Software;
- Computação, memória e rede e como este são monitorados;
- Como construir um Software? Código-fonte, compilador, código objeto, linker e executável;
- Linguagem de Programação e Paradigmas de Programação;
- Ciclo de Vida de Desenvolvimento de Software;
- CI / CD;
- Escopo, dependência e inversão de dependência;
- Acoplamento/desacoplamento, granularidade e coesão;
- Escalabilidade;
- Estudo de Caso.



IMPACTA DIGITAL